Implementação de um bate-papo utilizando o protocolo MQTT na linguagem Java

Erickson G. Müller

11 de Novembro de 2026

Capítulo 1

Introdução

O projeto de bate-papo com o protocolo MQTT foi desenvolvido em Java utilizando a biblioteca Paho para essa linguagem [1] e o broker Mosquitto [2]. A interface é manuseada por terminal através de comando indicados pelo próprio programa. O trabalho foi desenvolvido individualmente com o auxílio do modelo de linguagem Gemini Flash Lite de forma não-agêntica.

A proposta do trabalho foi desenvolver um sistema de bate-papo um-a-um e em grupo, possibilitando a listagem de usuários disponíveis e ausentes, listagem de grupos e seus respectivos líderes e membros, gerenciamento de grupos, notificações e histórico de ações no sistema de maneira cronológica.

[1] `https://eclipse.dev/paho/files/javadoc/index.html`
[2] `https://mosquitto.org/`

Capítulo 2

Instalação

Para compilar e rodar a aplicação, é necessário ter o **Java Development Kit (JDK)** e o broker **Eclipse Mosquitto** instalados. Certifique-se de que a biblioteca Paho (arquivo '.jar') está localizada dentro da pasta 'lib/' do projeto.

2.1 Dependências

```
Linux:
sudo pacman -S jdk-openjdk mosquitto
ou
sudo apt update
sudo apt install default-jdk mosquitto

MacOS:
brew install java mosquitto

Windows:
winget install Microsoft.OpenJDK.17
```

2.2 Compilação

A compilação do programa deve ser executada com o seguinte comando:

```
Linux e MacOS:
javac -cp "lib/*" -d bin src/chatmqtt/*.java

Windows:
javac -cp "lib\*" -d bin src\chatmqtt\*.java
```

2.3 Inicialização do Broker

Após as instalações necessárias, o programa necessita que você inicie o broker local para rotear as mensagens. Abra um terminal exclusivo e digite o seguinte comando:

```
Linux e MacOS:
mosquitto -v

Windows:
mosquitto.exe -v
```

2.4 Execução

Para a execução de cada cliente, inicie o programa com o seguinte comando:

```
Linux e MacOs:
java -cp "bin:lib/*" chatmqtt.chat

Windows:
java -cp "bin;lib\*" chatmqtt.chat
```

Capítulo 3

Arquitetura e Classes

O software foi desenvolvido com uma arquitetura modularizada, separando as partes de comunicação de rede, interface e lógica de estados. A aplicação é dividida em seis classes principais:

3.1 `controller.java`

Esta classe atua como a ponte de comunicação entre a aplicação e o broker Mosquitto, encapsulando os métodos da biblioteca Eclipse Paho MQTT. Suas principais responsabilidades são:

- Estabelecer a conexão TCP persistente com o servidor (utilizando a flag `setCleanSession(false)`).
- Assinar e desassinar tópicos conforme a criação e encerramento de conversas, para isso foram criados os métodos `assinarTopico` e `desassinarTopico`.
- Publicar mensagens padrão (com QoS 1) e mensagens retidas (para controle de status e registro em grupos).
- Implementar o *callback* assíncrono com o método principal (`messageArrived`), que atua como um roteador central. Ele intercepta todos os pacotes recebidos pela rede e os despacha para as classes de gerenciamento adequadas com base no prefixo do tópico.

Alguns métodos foram criados para automatizar a configurações que se repetem no gerenciamento do callback, como `desconectar(), enviarMensagem(), enviarMensagemRetida() e removerMensagemRetida()`.

Funções do CallBack:

```
public void connectionLost(Throwable cause) {...}
public void deliveryComplete(org.eclipse.paho.client.mqttv3.IMqttDeliveryToken token) {...}
public void messageArrived(String topic, MqttMessage message) throws Exception {
 String payload = new String(message.getPayload());
```

```
 if (topic.startsWith("USERS/")) {// Criar subtópicos para evitar o problema de
      // Guardar apenas uma mensagem
  gerenciadorUsuarios.atualizarStatus(payload);
 }
 else if (topic.startsWith("GROUPS/")){ // Cria subtópico GROUPS/
  gerenciadorGrupos.atualizarGrupo(topic, payload);
 }
 else if(topic.startsWith("CHAT_GRUPO/")){...}
 else if (
   !topic.startsWith( "USERS/") &&
   !topic.startsWith("GROUPS/") &&
   !topic.endsWith("_Control")
 ){...} // Chate privado. Encerrar conversa ou publicar mensagem
 // Ler mensagens que chegam em user_Control
 else if(topic.equals(userTopicoControle)){
  if(payload.startsWith("REQ:")){...}
  if(payload.startsWith("ACCEPT:")){...}
  if(payload.startsWith("REJECT:")){...}
  if(payload.startsWith("GROUP_REQ:")){...}
  if(payload.startsWith("GROUP_ACCEPT:")){...}
  if(payload.startsWith("GROUP_REJECT:")){...}
 }
}
```

Funções do Controller:

```
public void assinarTopico(String topico) throws MqttException {...}
public void desassinarTopico(String topico) throws MqttException {...}
public void publicarStatus(String status) throws MqttException {
 String messageStatus = userId + ":" + status;
 MqttMessage statusUser = new MqttMessage(messageStatus.getBytes());
 statusUser.setQos(1);
 statusUser.setRetained(true);// para quem logar depois
 client.publish("USERS/" + userId, statusUser); // Publica no subtópico
}
public void desconectar() throws MqttException {...}
public void enviarMensagem(String topicoDestino, String mensagem) throws MqttException{...}
public void enviarMensagemRetida(String topicoDestino, String mensagem)
         throws MqttException{...}
public void removerMensagemRetida(String topicoDestino) throws MqttException{...}
```

3.2 session.java

É o motor central de gerenciamento de estado das conversas e interações do usuário. A classe `session` não interage diretamente com o MQTT, mas mantém

toda a memória operacional da aplicação. Ela gerencia:

- **Histórico e Conversas Ativas:** Mantém na memória todas as salas de bate-papo abertas e as mensagens trocadas. `private Map<String, List<String>> historicoMensagens;` e `private Map<String, String> conversasAtivas;`
- **Controle de Tela :** Armazena a referência da "sala ativa" atual na tela do terminal. Se uma mensagem chega para a sala que o usuário está visualizando, ela é impressa instantaneamente; caso contrário, a classe emite uma notificação assíncrona não obstrutiva.
- **Solicitações:** Controla as filas (utilizando estruturas `HashSet` e `HashMap`) e os logs de pedidos pendentes de conversas privadas e de ingressos em grupos.

```
public void adicionarSolicitacao(String idSol) {...}
public void removerSolicitacao(String idSol) {...}
public boolean possuiSolicitacao(String idSol) {...}
public void listarPendentes() {
 System.out.println("\n--- Solicitações de Conversa ---");
 if (solicitacoesPendentes.isEmpty()) {
  System.out.println("Nenhuma solicitação pendente");
 }
 else{
  for (String solicitante : solicitacoesPendentes){
   System.out.println("- " + solicitante);
  }
 }
}
public void listarLogSolicitacoes() {
...
  for (String registro : logSolicitacoes){
   System.out.println(registro);
  }
...
}
public void registrarLogSolicitacoes(String tipo, String remetente, String destinatario,
String canal){
 String timestamp = LocalDateTime.now().format(formatter);
 String log = "["+tipo+"]"+timestamp+" " + remetente + "->" + destinatario;
 if (tipo.equals("ACEITO")){
  log += " ("+canal+")";
 }
 logSolicitacoes.add(log);
}
public void solicitarParticipacao(String membro, String grupo){
```

```
 solicitacoesGrupo.put(membro,grupo);
 System.out.println("\r\n[NOTIFICAÇÃO] Nova solicitação de participação ao grupo
 '"+grupo+"' vindo de "+membro );
 System.out.print(">> ");
}
public void removerSolicitacaoGrupo(String membro){...}
public boolean possuiSolicitacaoGrupo(String membro){...}
public String getGrupoSolicitado(String membro){...}
public void listarSolicitacoesPendentesGrupo(){
 System.out.println("Solicitações de participação Pendentes:");
 if (solicitacoesGrupo.isEmpty()){
  System.out.println("Nenhuma solicitação no momento");
  return;
 }

 for (Map.Entry<String, String> entry : solicitacoesGrupo.entrySet()){
  System.out.println("Usuário '"+entry.getKey()+"' solicitou participação no grupo '"
  +entry.getValue()+"'");
 }
}
public void adicionarConversa(String utilizador, String topico) {...}
public void encerrarConversa(String utilizador){...}
public String getTopicoConversa(String utilizador) {...}
public void listarConversasAtivas() {...}
public void setSalaAtiva(String topico) {...}
public void encerrarConversaPorTopico(String topico) {...}
public void registrarMensagemChat(String topico, String mensagem, String userId) {
...
 if (topico.equals(salaAtivaNaTela)) {
  if (!mensagem.startsWith("["+userId+"])) {
   System.out.println(mensagem);
  }
 }
 else {
  if (!mensagem.startsWith("["+userId+"])) {
   System.out.print("\r\n[NOTIFICAÇÃO] Nova mensagem recebida\n>> ");
  }
 }
...
}
public void registradaMensagemGrupo(String nomeGrupo, String mensagem, String userId){
 String topico = "CHAT_GRUPO/"+nomeGrupo;
 historicoMensagens.putIfAbsent(topico, new ArrayList<>());
 historicoMensagens.get(topico).add(mensagem);
...
}
```

```
public void imprimirHistorico(String topico) {
 List<String> hist = historicoMensagens.get(topico);
 if (hist != null && !hist.isEmpty()){
  for (String msg : hist) {
   System.out.println(msg);
  }
 }
}
```

3.3 groups.java

Classe voltada exclusivamente à manutenção das estruturas de dados das salas de chat coletivas. O sistema utiliza esta classe para recriar a estrutura hierárquica dos grupos na memória RAM de cada cliente. A classe mantém um dicionário (`Map`) que vincula o nome de cada grupo a um objeto interno chamado `infoGroup`. Este objeto registra estritamente qual é o ID do líder e um conjunto de dados (`Set`) com os IDs de todos os membros comuns, permitindo a validação rápida de permissões (como quem pode deletar o grupo ou enviar mensagens).

```
class infoGroup {
 String lider = "";
 Set<String> membros = new HashSet<>();
}

private Map<String, infoGroup> listaGroups;

public groups() {
 this.listaGroups = new HashMap<>();
}
public void atualizarGrupo(String topico, String payload){...}
public Set<String> listarMembros(String nomeGrupo){...}
public boolean esseGrupoExiste(String nomeGrupo){...}
public void listarGrupos(){...}
public String retornaLider(String nomeGrupo){...}
```

3.4 users.java

Responsável pelo controle de presença da aplicação. Processa as mensagens retidas vindas do tópico raiz `USERS/#` e atualiza instantaneamente a lista local informando quais clientes estão `ONLINE` ou `OFFLINE`. Isso garante que o menu de listagem de usuários reflita o estado real e sincronizado da rede em qualquer instante.

```
public users() {
 this.onlineUsers = new HashSet<>();
}
public void atualizarStatus(String mensagem) {...}
public void listarUsers() {...}
```

3.5 menu.java

Implementa a interface de linha de comando que o usuário utiliza para interagir com o programa. Suas funções abrangem:

- Capturar as entradas de teclado (através da classe `Scanner`) e processar a navegação nos submenus hierárquicos de conversas e grupos.
- Bloquear o fluxo contínuo de execução durante um bate-papo ativo (através de laços de repetição `while`) para permitir o envio de mensagens de texto sem a interferência do menu principal.
- Intermediar a lógica complexa de saída, abandono e exclusão de conversas e grupos, chamando coordenadamente os métodos de limpeza visual da classe `session` e as funções de exclusão de mensagens retidas da classe `controller`.

```
public void exibir() {
 boolean onscreen = true;
 while (onscreen) {
  System.out.println("1. Listar usuários");
  System.out.println("2. Conversas");
  System.out.println("3. Grupos");
  System.out.println("4. Histórico"); // Tem mais coisa aqui, abrir outro menu
  System.out.println("0. Sair");

  System.out.print(">> ");
  String opcao = scanner.nextLine();

  switch (opcao) {...}
 ...
}
private void subMenuConversas(){
 sessoes.listarPendentes();
 System.out.println("1. Solicitar nova conversa");
 System.out.println("2. Aceitar solicitação pendente");
 System.out.println("3. Recusar solicitação pendente");
 System.out.println("4. Mostrar registros de solicitações");
 System.out.println("5. Entrar numa conversa ativa");
 System.out.println("0. Voltar");
```

```
 System.out.print("Escolha: ");
...
  else if (subOpcao.equals("5")){ // Modo Chat
   sessoes.listarConversasAtivas();
   System.out.print("Digite o ID do utilizador com quem deseja conversar: ");
   String usuarioChat = scanner.nextLine();
   String topicoChat =  sessoes.getTopicoConversa(usuarioChat);

   if(topicoChat == null){
    System.out.println("Não existe conversa ativa com esse usuário.");
   }
   else {
    System.out.println("----------Conversa com: " + usuarioChat);
    System.out.println("          nome do canal: "+ topicoChat);
    System.out.println("\n Digite '/sair' para voltar ao menu ou '/encerrar' para fechar
    a conversa permanentemente");
    sessoes.setSalaAtiva(topicoChat);
    sessoes.imprimirHistorico(topicoChat);

    boolean onChat = true;
    while (onChat) {
     String texto = scanner.nextLine();
     if (sessoes.getTopicoConversa(usuarioChat) == null){
      // Se o outro usuario encerra a conversa com a tela aberta
      onChat = false;
      sessoes.setSalaAtiva(null);
      System.out.println("_________________________________________________");
      break;
     }

     if (texto.equals("/sair")) {
      onChat = false;
      sessoes.setSalaAtiva(null);
      System.out.println("_________________________________________________");
     }
     else if(texto.equals("/encerrar")){
      onChat = false;
      sessoes.setSalaAtiva(null);
      // Avisa o outro usuario que a conversa foi encerrada.
      mqttController.enviarMensagem(topicoChat, "CMD_ENCERRAR:"+userId);
      sessoes.encerrarConversa(usuarioChat);
      mqttController.desassinarTopico(topicoChat);
      System.out.println("Conversa com "+usuarioChat+" encerrada");
      System.out.println("_________________________________________________");
     }
     else{
```

```
      String mensagemChat = "["+userId+"]: "+ texto;
      mqttController.enviarMensagem(topicoChat, mensagemChat);
     }
    }

}
private void subMenuGrupos() {
 System.out.println("1. Listar grupos cadastrados");
 System.out.println("2. Criar novo grupo");
 System.out.println("3. Solicitar entrada num grupo");
 System.out.println("4. Gerenciar pedidos de entrada");
 System.out.println("5. Sair de um grupo");
 System.out.println("6. Entrar no chat de um grupo");
 System.out.println("0. Voltar");
 System.out.print("Escolha: ");
...
}
private boolean abandonarGrupo(String nomeGrupo) throws MqttException{...}
```

3.6 chat.java

Classe que possui o método main do código, define o broker como `tcp://localhost:1883`. Apresenta a tela de login e em seguida faz as chamadas de função para iniciar a sessão e assinar o tópico de controle do cliente e atualização de status.

Capítulo 4

Funcionalidades

4.1 Comunicação um-a-um

Quando um usuário deseja iniciar um bate-papo, ele envia uma solicitação que fica pendente até o aceite do destinatário. Após a aceitação, o sistema gera dinamicamente um tópico exclusivo para a sessão (ex: `Usuario1_Usuario2_Timestamp`), garantindo o isolamento da conversa. As mensagens de texto são enviadas com Qualidade de Serviço 1 (QoS 1 - *At least once*) e a conexão é configurada com `setCleanSession(false)`, o que permite que o broker Mosquitto enfileire mensagens caso um dos usuários perca a conexão ou fique offline temporariamente, entregando-as assim que a rede for restabelecida. O chat possui auto-recuperação de estado e um comando `/encerrar` para destruir a sessão e desassinar o tópico mutuamente.

4.1.1 Listagem de Usuários e Status

Para o controle de presença (online/offline), a aplicação utiliza mensagens retidas (*Retained Messages*) do protocolo MQTT. Assim que um cliente conecta ao broker, ele publica seu status no tópico `USERS/SeuID` com a flag de retenção ativada e carga útil (*payload*) de `ONLINE`. Na desconexão, o status `OFFLINE` é emitido. Como as mensagens são retidas, qualquer usuário que entrar no sistema posteriormente recebe instantaneamente o panorama atualizado de quem está disponível, gerenciado localmente via estruturas de dados `HashSet`.

4.2 Comunicação em grupo

O sistema permite a criação de salas de bate-papo coletivas de forma descentralizada. O usuário que cria o grupo é designado automaticamente como Líder. Usuários comuns podem listar os grupos existentes e enviar solicitações de entrada. O líder possui autonomia para aprovar ou recusar pedidos e é o único

com permissão para dissolver o grupo (eliminando a sala para todos os participantes). Membros comuns podem abandonar o grupo a qualquer momento através do comando `/abandonar` ou através do sub-menu de grupos. A troca de mensagens ocorre em um tópico dedicado com o prefixo `CHAT_GRUPO/`.

4.2.1 Listagem de Grupos

Assim como na listagem de usuários, a estruturação dos grupos tira proveito das mensagens retidas. A participação é registrada no broker sob o padrão `GROUPS/NomeDoGrupo/Usuario`, com a carga útil definindo o nível de permissão (`LEADER` ou `MEMBER`). Para remover um membro ou apagar um grupo inteiro, o sistema envia uma mensagem com *payload* vazio (zero bytes) para o respectivo tópico, sinalizando ao Mosquitto que a mensagem retida deve ser deletada, o que atualiza as listas locais de todos os clientes conectados.

4.3 Organização de Tópicos

Para manter a segregação lógica e evitar o cruzamento de mensagens de dados com mensagens de controle, a árvore de tópicos foi estruturada da seguinte maneira:

4.3.1 Tópico de controle do cliente

Cada usuário assina obrigatoriamente um canal particular com o sufixo `_Control` (ex: `Erickson_Control`). Este tópico é oculto e dedicado exclusivamente à sinalização do sistema. É através dele que transitam requisições (`REQ`, `GROUP_REQ`), aprovações (`ACCEPT`, `GROUP_ACCEPT`) e rejeições, funcionando como uma camada de controle separada da camada de texto dos chats.

4.3.2 Tópico de controle de grupos

O gerenciamento dos grupos é dividido em dois ramos principais:

- `GROUPS/#`: Tópico de administração assinado por todos os usuários de forma global. Usado estritamente para manter a lista de grupos, líderes e membros sincronizada na memória RAM de cada cliente.
- `CHAT_GRUPO/#`: Tópicos de comunicação efetiva. Apenas os membros validados de um grupo assinam o respectivo canal de texto, garantindo privacidade e evitando o recebimento de mensagens de grupos dos quais o usuário não faz parte.

4.4 Estrutura das mensagens

Para o correto roteamento e interpretação de eventos pela classe `controller`, os pacotes de dados (*payloads*) foram padronizados utilizando *strings* separadas

pelo caractere delimitador dois-pontos (`:`). Abaixo estão as estruturas implementadas no sistema:

- **Mensagens de Bate-papo:** Formatadas como `[ID_Remetente]: Texto_da_mensagem`. Aplicadas tanto no chat privado quanto nos grupos.
- **Solicitações Privadas:**
  - Pedido: `REQ:ID_Solicitante`
  - Aceite: `ACCEPT:ID_Destinatario:Topico_Gerado`*.
  - Rejeição: `REJECT:ID_Destinatario`

  ∗`Topico_Gerado` segue o padrão `ID1_ID2_Timestamp`
- **Gerenciamento de Grupos:**
  - Pedido de entrada: `GROUP_REQ:Nome_do_Grupo:ID_Solicitante`
  - Aceite de entrada: `GROUP_ACCEPT:Nome_do_Grupo`
  - Rejeição de entrada: `GROUP_REJECT:Nome_do_Grupo`
  - Comandos de exclusão mútua: `CMD_ENCERRAR_GRUPO` ou `CMD_ENCERRAR:ID_Remetente`.
- **Status de Presença e Hierarquia:** Publicações retidas simples, transmitindo apenas o valor final do estado, como `ONLINE` / `OFFLINE` para os usuários, e `LEADER` / `MEMBER` para os membros de grupos. Estruturando os tópicos de forma que o contexto da mensagem esteja na hierarquia, economiza-se o tráfego de rede através de mensagens menores.

4.5 Notificações

O recebimento de mensagens e eventos operacionais baseia-se num modelo totalmente assíncrono utilizando *Callbacks* (`messageArrived`) da biblioteca Eclipse Paho. Isso permite que a aplicação rode continuamente, aguardando interações do usuário no console principal, enquanto uma *thread* secundária intercepta pacotes do broker e imprime notificações de mensagens recebidas, pedidos de amizade e alertas de sistema na tela de forma não obstrutiva.

```
void setCallback(MqttCallback callback)

Sets the callback listener to use for events that happen asynchronously.

void messageArrived(java.lang.String topic,
                    MqttMessage message)
             throws java.lang.Exception
```